Information Security

Lecture 2b: Stateless Tracking and Browser Defences

A textbook treatment

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **2**

3 Stateless Tracking and Browser Defences **3**

3.1 Recognition Without State 3

3.2 Entropy: How Much a Feature Identifies 4

3.3 Attributes and Their Physics 5

3.4 Uniqueness, Linkability, Consistency 6

3.5 Blocking: Filter Lists and Behaviour 7

3.6 Browsers Against Tracking 8

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 3

Stateless Tracking and Browser Defences

3.1 Recognition Without State

Definition 3.1.1 (Fingerprint; stateful vs stateless)**.** A **browser fingerprint** is the set of information about a device, from hardware through OS to browser configuration, that a page can observe; **fingerprinting** is collecting it to recognise the device. **Stateful** tracking stores an identifier and reads it back (cookies, storage, advertising IDs): "I recognise you because I stored something". **Stateless** tracking recomputes a recognisable pattern from properties exposed *now*: "I recognise you from what you reveal each time". Clearing state does not stop recomputation.

Reading the formula

The test is *where the identifier comes from.* An ETag is stateful (it sits in the cache) though it is not a cookie; a canvas hash is stateless though it looks like an ID. The TP splits collection into *passive* (what the server receives with no code: IP, `User-Agent`, `Accept-*`, client hints), *active* (JavaScript queries APIs: `navigator`, `screen`, `window`, `Intl`, `document.referrer`) and *behavioural* (typing speed, key-hold time, inter-key interval, pauses, corrections; probabilistic, since keyboard, fatigue and stress change them). Features in a fixed order, hashed with SHA-256, give one short identifier (`/collect?fp=8b91c7a4...`) that the user cannot read, which is why fingerprinting is hard to notice.

The attack surface grew with the browser: Netscape Communicator 5.0 (1998, ≈742k lines) to Chromium 120 (January 2024, ≈31M lines); each new API (GPU, sensors, USB, Bluetooth, AR/VR) is a feature and a fingerprint attribute. A 1995 page saw "Netscape, French"; a 2024 page sees OS, screen, time zone, GPU model, audio and video capabilities.

Go deeper: the details an examiner asks about

- A fingerprint identifies a browser instance, never a named person (Quiz 1 Q11A false); exact matches have false positives.
- Quiz 1 Q13: before any JavaScript runs, the server observes IP, User-

Agent, Accept-Language, Accept-Encoding and client hints directly (A); screen and fonts need active collection (B false); headers do distinguish browsers (C false); no cookie is needed (D false).

- Disabling JavaScript stops active and behavioural collection; passive headers and stateful mechanisms remain.

Hard questions, answered

Q. Why is a hashed fingerprint not a permanent identifier?
A. Any changed feature (browser update alters the User-Agent, a new font) changes the hash; tracking over time needs *linking* algorithms (§3.4), not exact matching.

3.2 Entropy: How Much a Feature Identifies

Definition 3.2.1 (Surprisal and Shannon entropy)**.** If a feature takes value x_i with probability $P(x_i)$, observing x_i yields $-\log_2 P(x_i)$ bits of information (its *surprisal*). The expected information over the population is the entropy

$$H(X) = -\sum_i P(x_i) \log_2 P(x_i).$$

Low entropy: many users share the value; high entropy: the value is rare and narrows the population more.

Reading the formula

A value shared by half the population gives 1 bit (halves the candidates); one shared by 1/1000 gives ≈ 10 bits. Independent features add: $H(X,Y) = H(X)+H(Y)$. Singling out one of 7×10^9 devices needs $\log_2(7\times10^9) \approx 33$ bits. A uniform feature over n values has $\log_2 n$ bits (the maximum); a constant feature has 0. The distribution matters as much as the number of values: a skewed feature identifies less than a balanced one with the same support.

Worked example: Quiz 1, Q15 and a variant

100 fingerprints: 50 Windows, 25 Linux, 25 macOS.

$$H = 0.5\log_2 2 + 0.25\log_2 4 + 0.25\log_2 4 = 0.5 + 0.5 + 0.5 = 1.5 \text{ bits}$$

(answer C; the maximum for 3 values is $\log_2 3 \approx 1.585$). Time zone 80% UTC+1, 10% UTC+0, 10% UTC+2: $H = 0.8 \times 0.322 + 2 \times 0.1 \times 3.322 = 0.258 + 0.664 = 0.92$ bits: three values, less identifying than the OS, because skewed.

Hard questions, answered

Q. A feature has 16 equally likely values; another has 1000 values but one of them covers 99% of users. Which identifies more?
A. First: $\log_2 16 = 4$ bits. Second: $0.99\log_2(1/0.99) + 0.01\log_2(999/0.01) \approx 0.014+0.166 \approx 0.18$ bits on average (the rare values are informative but rarely

seen). Entropy is an expectation over the population.

Q. Why do fingerprint studies report entropy per attribute rather than "number of distinct values"?
A. Because identification power is the information observed, which depends on how values are distributed; 1000 values with one dominant is nearly useless, 16 balanced values are strong. Independence is needed to add attribute entropies; correlated attributes (OS and fonts) add less than their sum.

3.3 Attributes and Their Physics

- **HTTP headers / User-Agent**: passive; grew from `NCSA_Mosaic/2.0 (Windows 3.1)` to long strings naming engine, OS and version.
- **Screen, platform, time zone, hardware concurrency, fonts, extensions**: active; e.g. `3840x2160x24`, `Win32`, UTC+8, 4 cores.
- **Canvas** (Mowery & Shacham, W2SP 2012): JavaScript draws hidden text and shapes; the *instructions* are identical but the *rendered pixels* differ with browser, OS, fonts, GPU and graphics stack (anti-aliasing, hinting); read pixels, hash, use the hash as an attribute. Quiz 1 Q14: render predefined content, read pixels, hash (B); it stores nothing, retrieves no images, reads no serial number.
- **WebGL**: vendor and renderer strings ("NVIDIA, GeForce GTX 4070 Ti").
- **GPU timing** (DrawnApart, Laor et al., NDSS 2022): two machines identical in model, OS, browser and GPU model still differ because manufacturing variation makes some GPU execution units slightly faster; JavaScript measures timing traces; a deep-learning model compresses a trace into an embedding; same device gives a similar embedding despite noise. On 2,550 devices and 370,392 fingerprints over 7 months in the wild, adding it to FP-Stalker raised the median tracking duration (7-day revisit setting) from 17.5 to 28 days (+10.5, $\approx 60\%$). The only *hardware-level* attribute on the syllabus; it improves linkability, not uniqueness.
- **Battery Status API** (2012; Olejnik, Acar, Castelluccia, Diaz, *The Leaking Battery*, 2015): level, charging state, time remaining. Site A sees "73%, 14,237 s, not charging"; 20 seconds later site B sees "73%, 14,210 s": few devices match, and the drift is predictable, so the visits link *without any stored identifier*; the link expires as the battery drifts. A temporary signal for short-term linking; also abusable for price discrimination.
- **IP address**: passive; WWW'20: 87% of devices kept the same IPv4 for 30+ days (5,443 devices, 41,566 addresses), country/ISP dependent, the device–IP graph 93% unique; 20% show home–work cycles of 30+ days. Shared by NAT, so weak alone, strong combined.
- **Mobile**: also highly unique (S&P'16), vendor/model dependent, different attributes (User-Agent, emoji rendering, canvas).

Hard questions, answered

Q. Two identical laptops: which attribute separates them and why?
A. DrawnApart: GPU execution-unit timing reflects manufacturing variation,

invisible to every software attribute.

Q. Explain Battery-API linking and why it is not an identifier.
A. Level plus time-remaining plus charging state at time t is rare; at $t + 20$ s the same device shows the same level and ≈ 20 s less; probable match. Minutes later the combination has drifted and matches many devices: it links visits close in time only.

Q. Canvas needs JavaScript; User-Agent does not. Why?
A. Canvas requires executing drawing code and reading pixels back (active); the User-Agent is a header sent with every request (passive).

3.4 Uniqueness, Linkability, Consistency

Three research questions. **Uniqueness**: Panopticlick, 470,161 fingerprints, 94.2% unique; *Beauty and the Beast* (S&P'16), 118,934, 89.4%; *Hiding in the Crowd* (WWW'18), 1,816,776 desktop fingerprints, only 35.7% unique. **Linkability**, how fingerprints that *change* (updates, new fonts) can still be followed: FP-Stalker (S&P'18) tracks users for 55 days on average with the right linking algorithm; DrawnApart extends it; Blink (SEAMS'15) breaks it by generating a random disposable environment (OS, browser, fonts) per session; FPRandom (ESSoS 2017) adds controlled noise to canvas (adopted by Brave). **Consistency**: FP-Scanner (USENIX Security'18) shows most fingerprint-altering tools are detectable because attributes stop agreeing (User-Agent says Windows, fonts, platform and WebGL say Linux), and some tools leave per-user traces; the defence becomes a signal. Other works on the map: Fingerprinting in Style (USENIX Sec'21), Taming the Shape Shifter and Web Runner 2049 (DIMVA'20), Morellian analysis (DIMVA'19), FP-Redemption (DIMVA'21). `amiunique.org` (November 2014) shows your own fingerprint and its evolution.

Interpretation & discussion

Why did uniqueness fall from 94% to 36%? A far larger and more diverse sample; the disappearance of high-entropy attributes (Flash, Java, plugin lists); homogeneous populations on identical OS/browser versions. The conclusion is not "tracking stopped": 36% unique still leaves most browsers in small groups that an IP, a login or a battery signal collapses to one, and linkability over time does not need global uniqueness. Fingerprinting for *security* uses the same data defensively: "does this login look familiar?" compares the current browser to past logins of a known account; the fingerprint is a *risk signal*, never proof of identity (false positives and drift), so a mismatch triggers step-up authentication rather than denial.

Go deeper: the details an examiner asks about

- Quiz 1 Q11: Test 4 (all stores cleared, fingerprint 88 and profile 3 persist) is *consistent with* stateless relinking (B), and different browsers may share 88 (D); it proves neither a named identity (A) nor that clearing failed (C).
- Naive randomisation attacks linkability but creates *inconsistency*; a tool used by few people is itself a rare attribute. Brave's site-dependent noise

(§3.6) is the answer.

Hard questions, answered

Q. A defence randomises canvas on every load. Which property does it attack and what can go wrong?
A. Linkability (and the canvas attribute's uniqueness). Risks: FP-Scanner-style coherence checks detect the tool; the tool's rarity becomes an identifying attribute; sites that need stable canvas break.

Q. Why is 35.7% unique still a privacy problem?
A. Non-unique fingerprints lie in small groups; add IP or a short-term signal and the group is one device; and FP-Stalker-style linking follows a browser for weeks without uniqueness.

3.5 Blocking: Filter Lists and Behaviour

Definition 3.5.1 (Filter list; EasyList syntax)**.** A maintained set of rules describing requests or page elements a blocker should block, allow, redirect or hide: Disconnect (Firefox, Brave), EasyList (ads) and EasyPrivacy (tracking) (AdBlock Plus, uBlock Origin). A network rule `||tracker.com^` blocks every request whose URL matches: `||` anchors at the start of a domain (so `sub.tracker.com` matches, `nottracker.com` does not), `^` marks the end of the domain or a separator such as `/`. A rule beginning `@@` is an *exception* that allows something a broad rule would break.

Filter lists are *knowledge-based*: "we already know this domain tracks". Limitations: *coverage* (unlisted trackers load), *evasion* (rotating domains and URLs, first-party and CNAME techniques), *breakage* (login, video, payment, embeds), *maintenance* (continuous measurement and updates). **Behaviour-based** protection asks what a third party *does*: Privacy Badger (EFF) watches for identifying cookies, local-storage "supercookies" and fingerprinting, and restricts a third party seen tracking across several unrelated sites (the threshold trades false positives on ubiquitous CDNs against coverage). FP-detect inverts the problem: it detects which lists or extensions are active, so the blocker itself becomes an attribute.

Worked example: EasyList exercise

With `||amazon-adsystem.com^` present, both `/ad.js` and `/widgets/q?item=10` on that domain match the domain rule and are blocked *unless* an `@@` exception (e.g. for `/widgets/`) exists; the exercise's point is that exceptions exist to prevent breakage, so one checks the list rather than guesses. Rule "block all of `ads.example.net` except `/consent/`": `||ads.example.net^` plus the exception `@@||ads.example.net/consent/`.

Hard questions, answered

Q. A tracker is served from `static.publisher.com`. Which limitation, which protection?
A. Evasion by first-party/CNAME hosting; URL-based lists miss it. DNS-level inspection (Firefox resolves the CNAME chain) or behaviour-based de-

tection if it tracks across sites (it usually appears on one, so even Privacy Badger may miss it).

3.6 Browsers Against Tracking

- **Third-party cookie deprecation**. Google announced the end of third-party cookies in Chrome (January 2020): planned Q2 2022, delayed to Q4 2023, then Q4 2024; in 2024 Chrome moved to a *user-choice* approach; in 2025–2026 Google confirmed no universal phase-out. Lesson: browser privacy is shaped by compatibility, advertising economics, standards and deployment, not only by technique. Advertisers meanwhile built their own user IDs, device data and first-party data pipelines, and every platform opened a Conversions API (Meta, Amazon, Microsoft Offline Conversions, Google Enhanced Conversions, TikTok Events, X) to enrich first-party data.
- **Privacy Sandbox** (Chromium): Topics API (interest-based targeting), Protected Audiences (retargeting), Shared Storage (frequency capping), Attribution Reporting. *Topics*: each week the browser derives 5 topics from browsing history and adds one random topic, from a taxonomy of 469; the random topic gives deniability. Apple and Mozilla objected; PETS'24: the *sequence* of weekly topic sets observed by several sites is specific enough to serve as a quasi-identifier (stateless tracking built from a privacy API), and the design is "at odds with the natural diversity of user behaviours".
- **Firefox Total Cookie Protection**: a separate cookie jar per *top-level site*; a tracker embedded on A and B gets two different cookies, so Theorem 2.4.1 fails (it degrades to per-site analytics). Targets stateful cross-site tracking only; fingerprinting needs other defences; syncing within one site still works.
- **Brave randomisation**: subtle, *site-dependent* noise on selected fingerprintable outputs ("farbling"): the same site sees stable values (nothing breaks), different sites see different values (no linking); cites PriVaricator and FPRandom. Contrast FP-Scanner: inconsistent spoofing (one attribute changed, others untouched) reveals the tool and sometimes the real OS.
- **CNAME cloaking**: DNS `f0Ab1ze.mywebsite.com CNAME bad.tracker.com`; the browser sees a first-party hostname, cookies are first-party, URL filter lists miss it; only resolving the CNAME chain (Firefox exposes DNS-level APIs to blockers) reveals the tracker; a threat to transparency since the mapping lives server-side, and regulators (CNIL) must audit companies to see it.

Go deeper: the details an examiner asks about

- Dates to know: 2020 announce; 2022 Q2, 2023 Q4, 2024 Q4 planned; 2024 user choice; 2025–26 no phase-out.
- Quiz 1 Q6 under Total Cookie Protection flips: (A) and (D) become false, two jars, two `tid` values.
- Defence philosophies: isolate state (Firefox) vs randomise outputs (Brave) vs block known/behaving trackers (lists, Badger); a browser needs all three.

Hard questions, answered

Q. Why can Topics approximate an identifier although each week's set is only 5 of 469 topics plus a random one?
A. Several sites observe the same browser's weekly sets over many weeks; the sequence of sets is highly specific to one browsing history, so the sites can link the browser by matching sequences; the random topic adds noise but not enough across weeks.

Q. Why is CNAME cloaking "a big threat for transparency"?
A. Nothing in the page reveals the tracker: the hostname is first-party, the alias is in DNS and server configuration; users and URL-based blockers see a first-party request; only DNS inspection or an audit of the company exposes it.

Q. Compare Firefox partitioning and Brave randomisation: which threat does each address and what does each miss?
A. Firefox: stateful cross-site tracking via cookie reuse; misses fingerprinting, CNAME-cloaked first-party cookies, same-site syncing. Brave: fingerprint stability; misses stateful tracking unless combined with blocking/partitioning.

Summary: Stateless tracking and defences

- Stateless = recompute from exposed properties; passive / active / behavioural; a hash makes it one unreadable identifier of a *browser*, not a person.
- $H = -\sum P \log_2 P$; independent features add; 33 bits single out a device; skew lowers entropy.
- Attributes: headers, screen/fonts/time zone, canvas rendering, WebGL, GPU timing (hardware), battery (short-term), IP (stable weeks).
- Uniqueness fell to 36% on desktop; linkability (55 days, 28 with GPU) is the real threat; inconsistent spoofing is detectable.
- Lists (`||domain^`, `@@`) are knowledge-based and evadable (CNAME); Badger is behaviour-based; Firefox partitions, Brave randomises per site; Topics is a quasi-identifier; Chrome kept third-party cookies.